ER to Relational Mapping

Database Systems

Contents

1 ER Diagram Components **2**
- 1.1 Notation Reference . . . 2
- 1.2 Dual Notation: Chen and [min, max] . . . 2
- 1.3 [min, max] Reference . . . 3
- 1.4 Cardinality from [min, max] . . . 3
- 1.5 Exam Example: Doctor-Treat-Patient (Q3.2 Style) . . . 3
- 1.6 Exercise Example: Car Insurance ER Diagram . . . 4

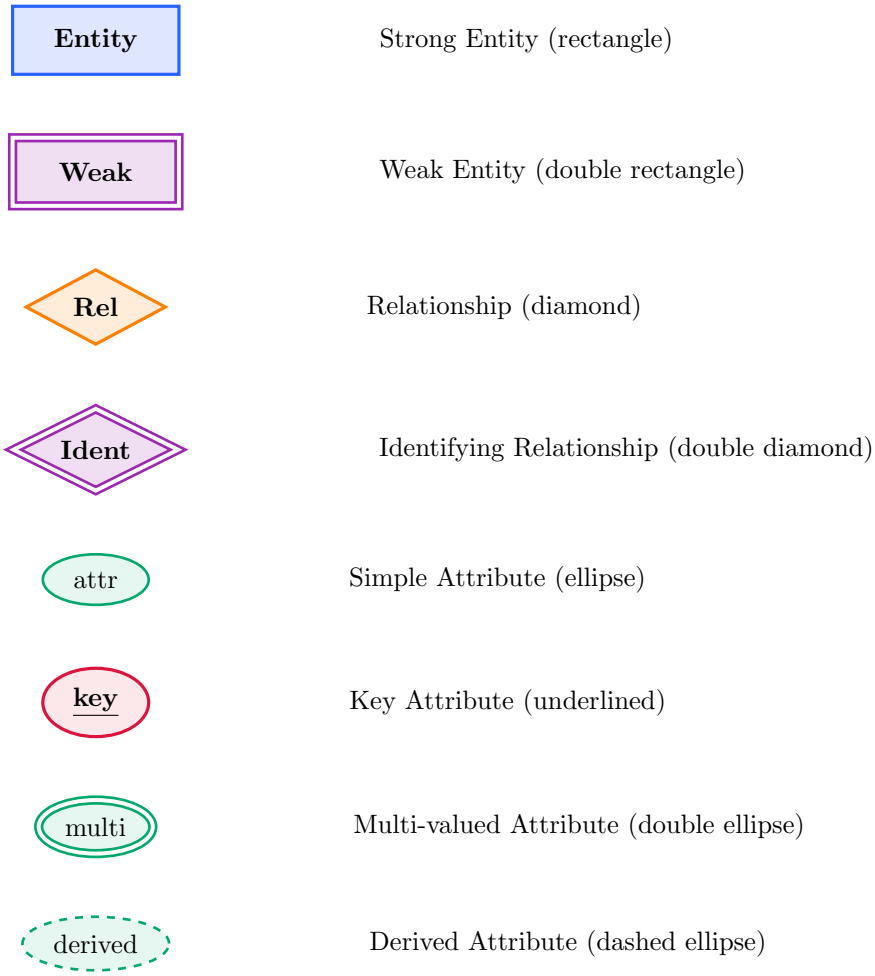
2 ER to Relational Mapping Algorithm **5**
- 2.1 Step 1: Strong Entity Types (M1) . . . 5
- 2.2 Step 2: Weak Entity Types (M2) . . . 5
- 2.3 Step 3: Binary 1:1 Relationships (M5) . . . 5
  - 2.3.1 Foreign Key Approach (Standard) . . . 6
  - 2.3.2 Merged Relation . . . 6
  - 2.3.3 Separate Relation . . . 6
- 2.4 Step 4: Binary 1:N Relationships (M4) . . . 6
- 2.5 Step 5: Binary M:N Relationships (M3) . . . 6
- 2.6 Step 6: Multi-valued Attributes (M7) . . . 7
- 2.7 Step 7: N-ary Relationships (n > 2) . . . 7
- 2.8 Exercise Example: Car Insurance Mapping . . . 7
- 2.9 Exam Example: Doctor-Belong-Department (Q3.3 Style) . . . 8

3 Special Cases and Common Patterns **10**
- 3.1 Composite Attributes . . . 10
- 3.2 Derived Attributes . . . 10
- 3.3 Multi-valued Attributes (Exam Pattern) . . . 10
- 3.4 Relationship Attributes . . . 11
- 3.5 Recursive (Self-Referencing) Relationships . . . 11
- 3.6 Participation and FK Constraints . . . 11
- 3.7 Common Exam Traps . . . 12
- 3.8 Weak Entity PK Pattern . . . 12

4 Reference Tables and Checklists **12**
- 4.1 Mapping by Relationship Type . . . 12
- 4.2 Attribute Placement Summary . . . 13
- 4.3 Schema Notation (Exam Format) . . . 13
- 4.4 Verbal Description to Notation . . . 13
- 4.5 FK Placement Quick Reference . . . 14
- 4.6 ER Diagram Construction Checklist . . . 14
- 4.7 Relational Mapping Checklist . . . 14
- 4.8 Shape Recognition Summary . . . 15
- 4.9 True/False Concept Quick Facts (Q3.1 Style) . . . 15

1 ER Diagram Components

1.1 Notation Reference



1.2 Dual Notation: Chen and [min, max]

ER diagrams use **two complementary notations** on connection lines:

Chen Notation (1

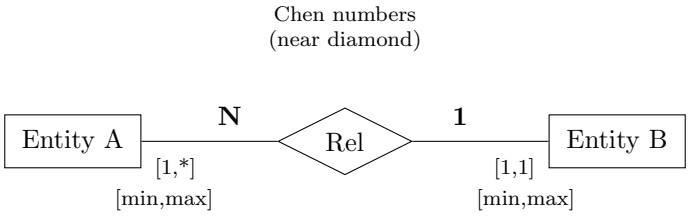
Numbers placed **near the relationship diamond**:

- **1** = at most one instance on this side
- **N** or **M** = many instances possible
- Reading: "For one instance of the OTHER entity, how many of THIS entity?"

[min, max] Notation

Constraints placed **near the entity**:

- **min**: minimum participation (0 = partial, ≥1 = total)
- **max**: maximum instances (1 or * /N)



1.3 [min, max] Reference

Notation	Meaning	Participation
[0, 1]	Optional, at most one instance	Partial
[1, 1]	Mandatory, exactly one instance	Total
[0, *]	Optional, unbounded (same as [0,N])	Partial
[1, *]	Mandatory, at least one (same as [1,N])	Total

Quick Rule

min = 0 ⇒ Partial participation (single line)
min ≥ 1 ⇒ Total participation (double line)

1.4 Cardinality from [min, max]

Entity A max	Entity B max	Cardinality	Type
1	1	**1:1**	One-to-One
1	*	**1:N**	One-to-Many
*	1	**N:1**	Many-to-One
*	*	**M:N**	Many-to-Many

*The **max** value determines cardinality ratio; the **min** value determines participation.*

1.5 Exam Example: Doctor-Treat-Patient (Q3.2 Style)

Scenario: Each patient must be treated by exactly one doctor (total participation). Each doctor can treat many patients or no patient (partial participation).

Reading the notation:

- **Patient [1,1]**: Each patient *must* be treated (min=1) by *exactly one* doctor (max=1)
- **Doctor [0,*]**: Each doctor *may* treat none (min=0) or *many* patients (max=*)
- **Chen N near doctor**: One patient → 1 doctor, but one doctor → N patients

1.6 Exercise Example: Car Insurance ER Diagram

The following diagram shows the teacher's solution for the Car Insurance Company ER diagram (Exercise 5, L05).

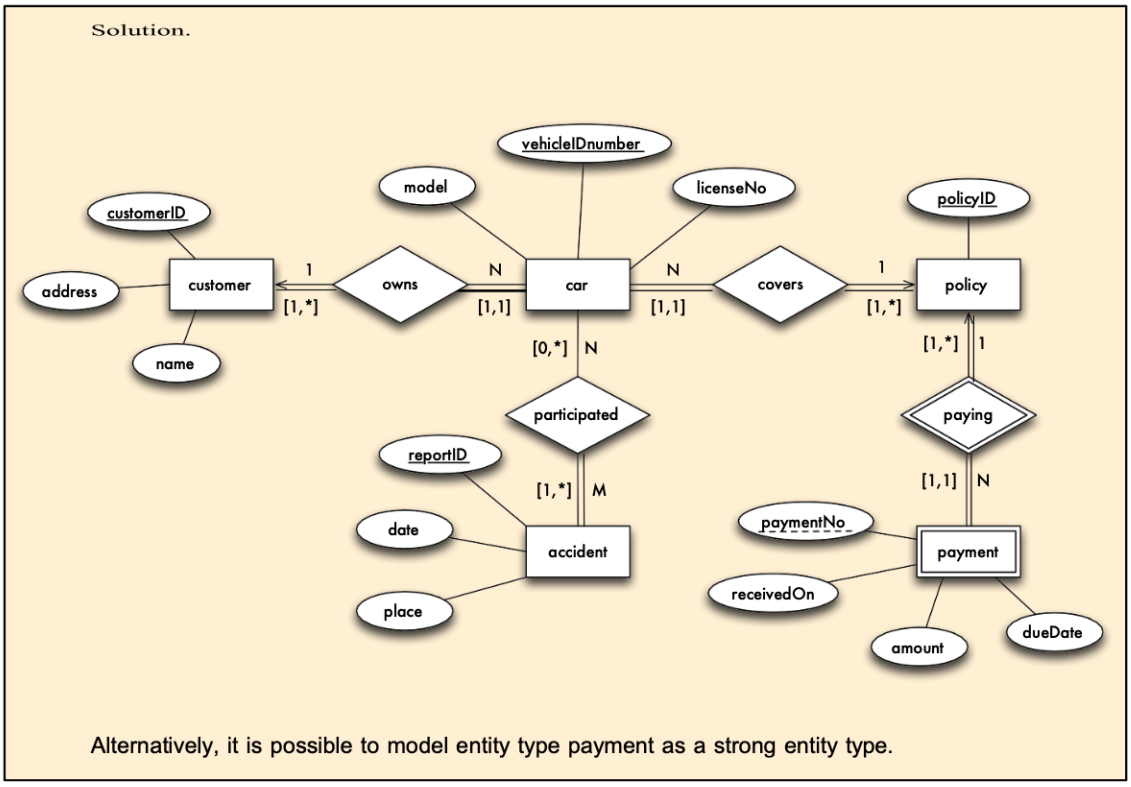


Figure 1: Car Insurance ER Diagram – Exercise 5 Solution (L05-ermodel)

Key observations from this diagram:

- **5 entities**: customer, car, policy, payment, accident
- **Weak entity**: payment (double rectangle) with identifying relationship "paying" (double diamond)
- **Partial key**: paymentNo has dashed underline (partial key for weak entity)
- **Chen notation**: 1, N, M numbers placed near diamonds
- **[min,max]**: constraints placed near entities
- **M:N relationship**: "participated" between car and accident

2 ER to Relational Mapping Algorithm

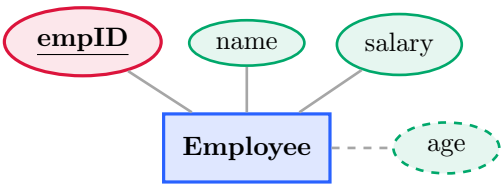
The 7 Mapping Steps

1. Map **strong (regular) entity types**
2. Map **weak entity types**
3. Map **binary 1:1 relationships**
4. Map **binary 1:N relationships**
5. Map **binary M:N relationships**
6. Map **multi-valued attributes**
7. Map **N-ary relationships** (n > 2)

2.1 Step 1: Strong Entity Types (M1)

For each **strong entity** E:

- Create a relation R with all **simple attributes** of E
- For **composite attributes**, include only the simple components
- Designate one key attribute as the **primary key** (PK)
- Exclude: derived attributes, multi-valued attributes

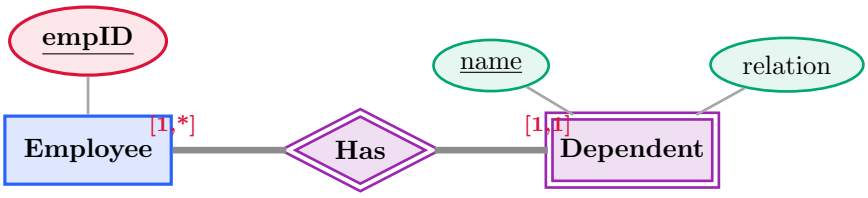


⇒ **Employee**(empID, name, salary)

2.2 Step 2: Weak Entity Types (M2)

For each **weak entity** W with owner entity O:

- Create relation R with all simple attributes of W
- Add the **primary key of the owner** O as a **foreign key** (FK)
- **Primary key of** R = FK (owner's PK) + partial key of W



Result: Dependent(empID–¿Employee, name, relation)

2.3 Step 3: Binary 1:1 Relationships (M5)

Three approaches based on participation:

2.3.1 Foreign Key Approach (Standard)

- Add PK of one entity as FK in the other entity's relation
- Place FK on the side with **total participation** (min ≥ 1)
- This minimizes NULL values

Employee(<u>empID</u>, name, ...)
Dept(<u>deptID</u>, deptName, managerID–¿Employee)

2.3.2 Merged Relation

If **both** entities have total participation, they can be merged into one relation.

2.3.3 Separate Relation

Create a new relation with both PKs. Applicable when both sides have partial participation.

2.4 Step 4: Binary 1:N Relationships (M4)

Add the PK of the "one" side as FK to the "many" side.

1:N

Result:

- Dept(<u>deptID</u>, deptName)
- Employee(<u>empID</u>, name, deptID–¿Dept)

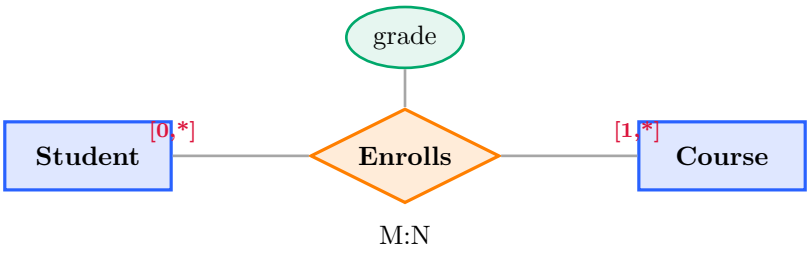
1:N Rule

In 1:N relationships, the FK **always** goes on the N-side (many side).
Relationship attributes also go with the FK (on N-side).

2.5 Step 5: Binary M:N Relationships (M3)

Create a new **relationship relation** containing:

- PKs of both participating entities as FKs
- These FKs form the **composite primary key**
- Any attributes of the relationship



Result:

- Student(stuID, name)
- Course(courseID, title)
- **Enrolls**(stuID–¿Student, courseID–¿Course, grade)

2.6 Step 6: Multi-valued Attributes (M7)

For each **multi-valued attribute** A of entity E:

- Create a **new relation** R
- Include attribute A and the PK of E as FK
- PK of R = combination of A and FK (composite key)

2.7 Step 7: N-ary Relationships (n > 2)

For relationships involving **more than two entities**:

- Create a new relation R
- Include PKs of **all participating entities** as FKs
- PK determination depends on cardinality constraints
- Include any relationship attributes

N-ary Primary Key

- All sides "many" → PK = all FKs combined
- One side "one" → that FK may not be part of PK

2.8 Exercise Example: Car Insurance Mapping

The following shows the teacher's solution mapping the Car Insurance ER diagram to relations (Exercise 6, L06).

Complete Relational Schema:

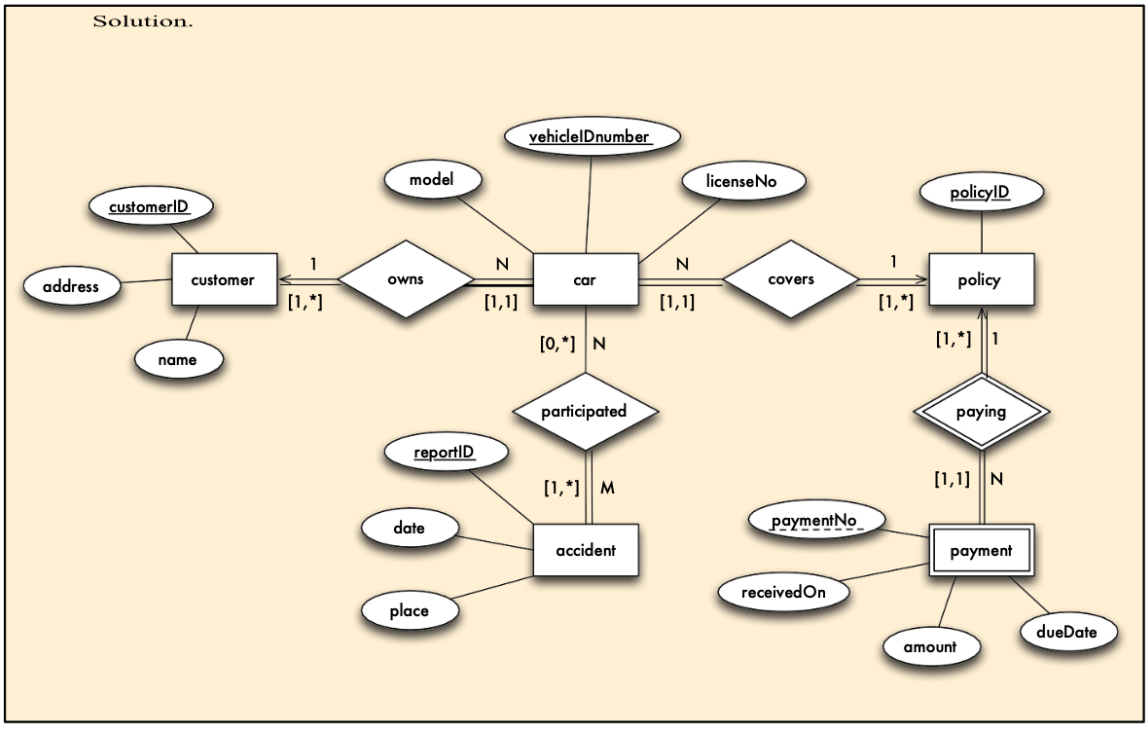


Figure 2: Car Insurance ER-to-Relational Mapping – Exercise 6 Solution (L06-ermodel-2)

Car Insurance Schema (6 Relations)

```
customer(customerID, address, name)
car(vehicleIDnumber, model, licenseNo, customerID-->customer,
    policyID-->policy)
policy(policyID)
payment(policyID-->policy, paymentNo, receivedOn, amount, dueDate)
accident(reportID, date, place)
participated(vehicleIDnumber-->car, reportID-->accident)
```

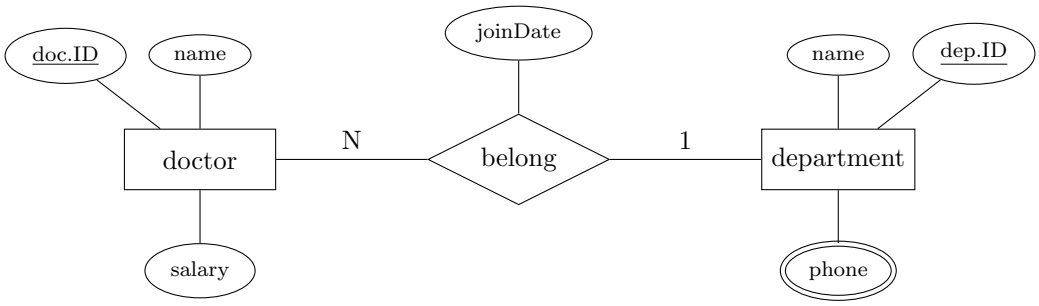
Mapping Analysis:

- **Strong entities** (customer, car, policy, accident): Direct mapping with PKs
- **Weak entity** (payment): Composite PK = policyID + paymentNo
- **1:N owns**: FK customerID added to car (N-side)
- **1:N covers**: FK policyID added to car (N-side)
- **M:N participated**: New relation with composite PK

2.9 Exam Example: Doctor-Belong-Department (Q3.3 Style)

Scenario from exam: Map the doctor-belong-department ER diagram where:

- Each doctor belongs to exactly one department (N:1)
- The "belong" relationship has attribute **joinDate**
- Department has multi-valued attribute **phone**



Correct Mapping (3 relations):

Doctor-Department Schema

```
doctor(doc.ID, name, salary, joinDate, dep.ID-->department)
department(dep.ID, name)
phone(dep.ID-->department, phoneNumber)
```

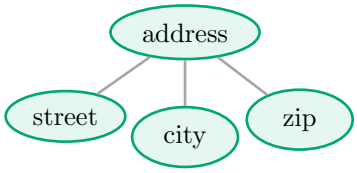
Key decisions:

1. **1:N belong**: FK (dep.ID) goes to doctor (N-side)
2. **Relationship attribute**: joinDate goes with the FK (in doctor table)
3. **Multi-valued phone**: Separate table with composite PK (dep.ID + phoneNumber)
4. **No separate belong table**: Inefficient for 1:N relationships

3 Special Cases and Common Patterns

3.1 Composite Attributes

Flatten by including only the **leaf-level simple attributes**:



⇒ Include: street, city, zip

3.2 Derived Attributes

Derived attributes are **not included** in the relational schema. They are computed from other stored attributes.

Examples:

- `age` derived from `birthDate`
- `totalPrice` derived from `quantity` × `unitPrice`

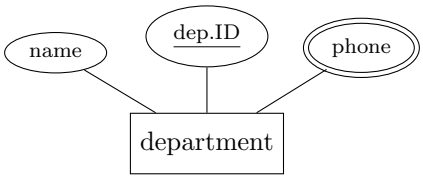
3.3 Multi-valued Attributes (Exam Pattern)

Multi-valued attributes **always** require a separate table.

Multi-valued Attribute Rule

1. Create a **new relation** for the multi-valued attribute
2. Include the **parent entity's PK** as a foreign key
3. The **composite PK** = parent's PK + attribute value
4. Both attributes in the PK must be **underlined**

Example from Q3.3 (department.phone):



Mapping Result:

```
department(dep.ID, name)            -- simple attrs only
phone(dep.ID-->department, phoneNumber)  -- separate table
```

In the phone table:

- dep.ID is both FK and part of composite PK
- phoneNumber is part of composite PK
- Both are underlined in exam notation

3.4 Relationship Attributes

Where relationship attributes go depends on the cardinality:

Cardinality	Attribute Placement	Reason
1:N	With FK (on N-side entity)	Only one value per N-side instance
M:N	In relationship relation	Multiple values possible
1:1	With FK (either side)	Only one value per pair

Example: joinDate in doctor-belong-department (1:N)
Since "belong" is a 1:N relationship (many doctors to one department):

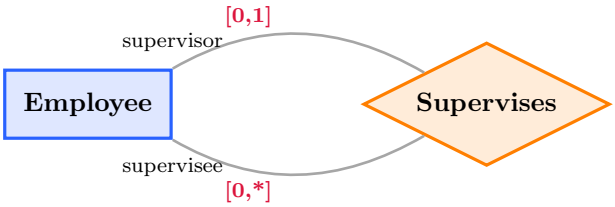
- The FK (dep.ID) goes to doctor (N-side)
- The relationship attribute (joinDate) goes **with the FK**
- Result: `doctor(doc.ID, name, salary, joinDate, dep.ID-->department)`

Why Not a Separate Table?

Creating a separate `belong(doc.ID, dep.ID, joinDate)` table for a 1:N relationship is **inefficient**. Since each doctor belongs to exactly one department, the FK and relationship attribute fit naturally in the doctor table.

3.5 Recursive (Self-Referencing) Relationships

The same entity participates multiple times with different **roles**:



Result: Employee(empID, name, supervisorID–¿Employee)
The FK references the **same table** (self-referencing).

3.6 Participation and FK Constraints

Participation	Min Value	FK Constraint
Total	min ≥ 1	FK is **NOT NULL**
Partial	min $= 0$	FK **can be NULL**

3.7 Common Exam Traps

Avoid These Mistakes

1. **Multi-valued in parent table**: Never include multi-valued attributes directly in the entity's table → always create separate table
2. **Separate table for 1:N**: Creating a relationship table for 1:N is inefficient → use FK approach
3. **FK on wrong side**: In 1:N, FK goes on N-side, not 1-side
4. **Missing composite PK underlines**: For M:N and multi-valued tables, **both** FK attributes must be underlined
5. **Relationship attr in wrong place**: In 1:N, relationship attribute goes with FK (N-side), not in a separate table

3.8 Weak Entity PK Pattern

For weak entities, the primary key is **always composite**:

$$\textbf{PK} = \text{Owner's PK} + \text{Partial Key (discriminator)}$$

Example: payment weak entity

```
payment(policyID-->policy, paymentNo, receivedOn, amount, dueDate)
```

Here:

- policyID is FK to owner AND part of composite PK
- paymentNo is partial key (discriminator) AND part of composite PK
- Both are underlined (composite PK)

4 Reference Tables and Checklists

4.1 Mapping by Relationship Type

Side A	Side B	Type	Mapping
[1,1]	[1,1]	1:1 total-total	Merge or FK either side
[0,1]	[1,1]	1:1 partial-total	FK on B (total side)
[0,1]	[0,1]	1:1 partial-partial	FK either side or separate relation
[1,1]	[1,*]	1:N total-total	FK on B (N-side)
[1,1]	[0,*]	1:N total-partial	FK on B (N-side), nullable
[0,1]	[1,*]	1:N partial-total	FK on B (N-side)
[0,*]	[0,*]	M:N	New relation, composite PK
[1,*]	[1,*]	M:N total-total	New relation, composite PK
[0,*]	[1,*]	M:N mixed	New relation, composite PK

4.2 Attribute Placement Summary

Attribute Type	Placement
Entity attribute (simple)	In the entity's relation
Entity attribute (composite)	Flatten: simple components only
Entity attribute (multi-valued)	Separate relation with composite PK
Entity attribute (derived)	**Exclude**
Relationship attribute (1:1, 1:N)	With the FK (in appropriate relation)
Relationship attribute (M:N)	In the relationship relation

4.3 Schema Notation (Exam Format)

Standard notation used in exams:

Schema Notation Format

`TableName(primaryKey, attr1, attr2, fkAttr-->ReferencedTable)`

Rules:

- Underline indicates primary key attribute(s)
- Foreign keys use arrow: `attr-->ReferencedTable`
- Composite PK: multiple underlined attributes
- Table names in lowercase (matching entity names)

Example Schema:

```
customer(customerID, address, name)
car(vehicleIDnumber, model, licenseNo, customerID-->customer,
    policyID-->policy)
policy(policyID)
payment(policyID-->policy, paymentNo, receivedOn, amount, dueDate)
accident(reportID, date, place)
participated(vehicleIDnumber-->car, reportID-->accident)
```

4.4 Verbal Description to Notation

Verbal Description	[min,max]	Chen
"must" / "exactly one"	[1,1]	1
"can" / "may" / "zero or more"	[0,*]	N
"at least one" / "one or more"	[1,*]	N
"at most one" / "optional one"	[0,1]	1

Example interpretation:

- "Each patient **must** be treated by **exactly one** doctor" → patient: [1,1]
- "Each doctor **can** treat **many** patients or **no** patient" → doctor: [0,*]

4.5 FK Placement Quick Reference

Cardinality	FK Goes To	Why
1:1	Total participation side	Minimizes NULLs
1:N	N-side (many side)	Each "many" points to "one"
M:N	New relation	Both FKs form composite PK
Weak entity	Weak entity table	Owner's PK becomes part of weak's PK

4.6 ER Diagram Construction Checklist

Before Submitting ER Diagram

- ☐ All entity types identified (rectangles)
- ☐ Weak entities have double rectangle + double diamond
- ☐ 2-3 attributes per entity (as specified)
- ☐ Primary keys underlined (solid line)
- ☐ Partial keys have dashed underline
- ☐ All relationships identified (diamonds)
- ☐ Chen notation (1, N, M) near diamonds
- ☐ [min, max] notation near entities
- ☐ Double lines for total participation
- ☐ Multi-valued attributes have double ellipse
- ☐ Derived attributes have dashed ellipse

4.7 Relational Mapping Checklist

Before Submitting Relational Schema

- ☐ Each strong entity → one relation
- ☐ Each weak entity → relation with owner's PK as FK
- ☐ Weak entity PK = owner's PK + partial key (both underlined)
- ☐ 1:N relationships: FK on N-side
- ☐ M:N relationships: new relation with composite PK
- ☐ Multi-valued attributes: separate relation
- ☐ Relationship attributes placed correctly
- ☐ All PKs underlined
- ☐ FK notation: `attr-->ReferencedTable`
- ☐ No redundant tables (minimal schema)
- ☐ Derived attributes excluded

4.8 Shape Recognition Summary

Shape	Border	Meaning
Rectangle	Single	Strong Entity
Rectangle	Double	Weak Entity
Diamond	Single	Regular Relationship
Diamond	Double	Identifying Relationship
Ellipse	Single, solid underline	Primary Key Attribute
Ellipse	Single, dashed underline	Partial Key
Ellipse	Single, no underline	Simple Attribute
Ellipse	Double	Multi-valued Attribute
Ellipse	Dashed	Derived Attribute
Line	Single	Partial Participation
Line	Double/thick	Total Participation

4.9 True/False Concept Quick Facts (Q3.1 Style)

Common True/False Statements

- **TRUE**: ER model is used during conceptual design phase
- **TRUE**: Dashed ellipse represents derived attribute
- **FALSE**: All attributes must be single-valued (multi-valued exist)
- **TRUE**: Primary key uniquely identifies entity instances
- **TRUE**: Weak entity depends on owner entity for identification
- **FALSE**: M:N relationships don't need a separate table (they do)